

Vroomie

Detection Architecture v10

Two-Tier Acoustic Matching

Complete technical reference for architects and development teams extending the
Vroomie audio anomaly-detection system

Engine version v10.0 · FINGERPRINT + EMBEDDINGS
github.com/DGeorgeA/vroomie-app · branch main
Live at vroomie.in

All performance figures herein are measured results on held-out data, reproducible with the committed test harnesses.

1. The one-page mental model

Vroomie now runs **two matchers with opposite strengths over one capture pipeline**:

```

Microphone (AGC/NS/EC requested off; actual settings recorded)
→ native-rate capture → 1 s ring buffer (~0.9 s cadence) → resample 16 kHz

├── TIER 1 – CONSTELLATION FINGERPRINT (Shazam-style) [≤5 s, exact recordings]
│   ├── rolling 5 s window → spectral-peak hashes → time-offset coherence
│   ├── HIT identify + auto-finalise session (97%, matchMethod:
│   │   │   acoustic_fingerprint). No hit completely silent.
│   └──
└── TIER 2 – EMBEDDING ENGINE (YAMNet) [generalising, session-based]
    ├── silence gate → RMS-normalize 0.05 → YAMNet → domain gate
    ├── → cosine vs 352 fault embeddings + 94 anchors → margin rule
    ├── → session vote (primary fraction / recovery vote)
    └── → possibility 70–97% statement
  
```

	Tier 1: Constellation	Tier 2: Embeddings
Question answered	"Is this exact recording playing?"	"Does this sound like a known fault family?"
Latency	~3–5 s, auto-finalises	full session (user stops)
Generalisation	none — by design	the whole point
Bench replay of a Supabase sample	primary owner	fallback
Real customer vehicle	blind (no shared hashes)	primary owner
Decision statistic	time-offset coherent votes	cosine margin over anchors
False-positive defence	coherence itself (unrelated audio can't align)	anchors + domain gate + session vote

The tiers are **additive**: Tier 1 never suppresses Tier 2; an unavailable fingerprint index simply means Tier 2 runs alone (fail-safe measured, not assumed).

2. Why a percentage threshold cannot live in Tier 1

The product question "if a squeak or bearing on a *different car* sounds similar, shouldn't ~65–72% match categorize it?" decomposes onto the two tiers:

- **Constellation scores are not similarity percentages.** A coherent score of 1400 vs 237 is not "6× more similar" — it counts hash pairs that agree at one time offset. Two *different recordings* of the same fault share almost **zero** hashes: this is the same property that makes Shazam identify a studio track but fail on a cover version. There is no threshold at which Tier 1 recognises a different vehicle — 65%, 72% or any other number. Lowering its thresholds does not add generalisation; it only admits random coherence (noise).
- **The 65–72% band already has a home: Tier 2's margin scale.** The published possibility is $70 + 108 \times (\text{margin} - 0.04)$ clamped to [70, 97] — so "65–72%" corresponds to margins just under the 0.04 confirm threshold. That is where a "possible match" tier belongs, and where it was measured (§6).

Rule for future tuning: route exact-replay demands to Tier 1, similarity demands to Tier 2. Never tune one tier to do the other's job — each failure mode of the past (hallucination, threshold-loosening, 20% healthy FP) came from asking a similarity system to behave like an identifier or vice versa.

3. Tier 1 in detail (what shipped, and the review that changed it)

Algorithm (`src/lib/constellationMatcher.js`, self-contained, no imports): 1024-pt FFT, 16 ms hop; per-band strongest bin above $1.6\times$ band mean (6 log-spaced bands); anchor–target pairing (fan-out 6, $\Delta t \leq 48$ frames) hashed as $(f1, \Delta f, \Delta t)$; matching by per-reference **offset histogram** — the score is the tallest single-offset bin. The index builder **imports the same hashing function**, so index/query parity is structural, not conventional.

Index (`public/constellation_v1.json`): 9 references (6 originals extended to 10 s + intake/misfire/timing at 5 s), 166 k entries, 1.7 MB, service-worker cached. The 10 s extensions use **equal-power crossfade looping** (`scripts/extend_reference_wavs.py`) because naive concatenation produces click transients that become spurious spectral peaks.

Runtime behaviour: rolling 5 s listener fed un-normalized 16 kHz audio (peaks are relative maxima — already level-invariant); earliest fire at 3 s; a hit toasts "Identified: X — matched in Ns", auto-finalises the session, bypasses abort gates (the audio was identified), and stores `matchMethod: 'acoustic_fingerprint'` with the coherence score.

3.1 Pre-ship review — flaws found and fixed

#	Flaw (measured)	Fix (measured)
F1	Threshold provenance : thresholds derived on a 6-ref prototype; against the <i>shipped</i> 13-ref index the worst healthy negative scored 443 — above the 400 raw floor (was 237).	Review protocol re-measures through the shipped hydrate/match code path. Floor raised 400 → 600 ; weakest passing positive condition is 621, so nothing that worked stops working; headroom restored to $\geq 1.4\times$ on both criteria.
F2	Short-reference bias : normalizing by query hashes alone means a 1.5 s reference physically cannot reach 0.05 with a 5 s listen — a genuine replay of an indexed 1.5 s file scored 385/0.035: failed against itself .	Normalization corrected to <code>score / min(queryHashes, refHashes)</code> ; per-ref <code>hash_count</code> now ships in the artifact. Additionally a measured 4 s minimum reference length — the four 1.5 s PS variants were dropped (their replay detection falls to Tier 2, which measurably handles them).
F3	Single-play vs looped : does one playback of a 5 s file inside the listen window still match?	Measured: intake 1574/0.144, misfire 676/0.062, timing 766/0.071 — all pass identically single-played or looped.
F4	Ambiguity from near-duplicate references	Measured non-issue: second-best scores 65–118 vs best 1388–2516.

3.2 Tier 1 operating point (current, all re-measured on the shipped index)

```

MIN_COHERENT_SCORE    = 600      worst negative 443 → 1.4× headroom
MIN_NORMALIZED_SCORE  = 0.05     worst negative 0.040 → 1.25× headroom
                                weakest passing positive: 621 / 0.06
                                strongest negative set: 115 healthy clips,
                                4 real speech recordings (direct + speaker-
                                colored), music, fan, traffic
LISTEN_SECONDS = 5, earliest fire 3 s

```

In-app proof (served build, speaker-colored audio injected as microphone): Piston → "Identified: Piston" at 3 s, auto-finalised, 97%; speech → no hit, correct diagnostic abort. Post-recalibration playback modes of the 2 s original: looped 1211/0.109, single-play 947/0.213, extended-10 s excerpt 4952/0.449 — all comfortably above the 600/0.05 thresholds.

4. Tier 2 in detail (the generalising engine, v9.9)

Unchanged by the constellation work — summarised for completeness; full history in

`VROOMIE_ARCHITECTURE_HANDBOOK.md`.

- **References:** offline factory (`build_reference_fingerprints.mjs`) → 352 embeddings, per-family cap 100 (power_steering was 58.8% of the set; the cap measurably cut healthy FPs 9→6/140 and fixed two speaker-coloration misclassifications), 8 augmentation variants including the harsh phone-speaker channel; 94 anchors (healthy + interferers incl. traffic).
- **Gate:** vehicle top-1 | weak generic | vehicle-evidence | **speaker-rescue** (interferer top-1 with vehicle evidence ≥ 0.02 and interferer ≤ 0.30 — measured 29/70 vetoed fault windows recovered, 0/108 speech/music admitted).
- **Match:** cosine ≥ 0.45 **and** margin over best anchor ≥ 0.04 .
- **Session:** primary fraction rule ($\geq 45\%$ of accepted windows, one family, ≥ 3 windows) + v9.8 recovery vote ($\geq 60\%$ total candidates → plurality; similarity-sum tie-break at 1.10 dominance, else decline).
- **Diagnosability:** every abort persists a hidden `status: 'rejected'` row with per-stage counts, what YAMNet heard, and the device's applied capture settings.

5. Implications of constellation matching for application finetuning

1. **Bench-testing is no longer a calibration signal for Tier 2.** Replayed Supabase samples will be captured by Tier 1 almost immediately. Any future "the sample isn't detected" report first asks: did Tier 1 miss (index/threshold issue — check `matchMethod` and coherence scores) or did it correctly fall through (capture too degraded — the diagnostics row tells you)? **Do not retune Tier 2 thresholds from bench replays anymore** — they will rarely even reach it.
2. **The reference library's two roles have diverged.** For Tier 1 a reference is a *recording identity* — longer is strictly better (more frames → taller coherence spike), and near-duplicates add nothing. For Tier 2 a reference is a *class sample* — diversity matters, duplicates skew the prior (the 58.8% dominance problem). Curate accordingly: one long canonical recording per identity for Tier 1; many *distinct* real recordings per family for Tier 2.
3. **Adding a new fault class** now takes three steps, all automated after upload: put a ≥ 4 s (ideally ≥ 10 s) real recording in the bucket → `node scripts/build_constellation_index.mjs` → `node scripts/build_reference_fingerprints.mjs`; then re-run `review_constellation_shipped.mjs` and the QA suites. The 4 s index minimum is enforced automatically; sub-4 s files still serve Tier 2.
4. **Index growth needs threshold re-validation.** F1 is a standing lesson: negative scores grow with index size (more chances of random alignment). The review script *is* the gate — re-run it whenever `reference_count` changes, and treat its worst-negative line as the floor-setter.
5. **Latency identity.** Tier 1 gives the product its "Shazam moment" (~3 s). Nothing in Tier 2 should be tuned to chase that number — its value is generalisation, and its honest latency is a session. The UI already reflects this split (instant "Identified" toast vs end-of-session report).
6. **What still cannot be finetuned into existence:** recognising a *different* vehicle's similar-sounding fault at high confidence. That requires data (real phone-mic recordings of real faulty vehicles feeding Tier 2), not

thresholds. This remains the single highest-leverage investment in the product.

6. The 65–72% "possible match" tier (Tier 2 extension)

Semantics: a window is a *near* candidate when similarity ≥ 0.45 and margin $\in [0.02, 0.04)$ — i.e. the sound is much closer to a fault family than to any healthy/noise anchor, but below the confirmed band. A session reports **"possible $\square$ family $\square$ — verification required"** at 65–69% only when nothing was confirmed and near+full candidates of one family reach the same 45% fraction.

Measured feasibility (scripts/measure_near_match_tier.mjs, 140 held-out healthy clips + known acoustic misses + interferer suite):

Axis	Result
Value — known acoustic misses recovered	0 of 3. Rocker@2m and MotorStarter now confirm outright under v9.9 (the tier is moot for them); the alternator-critical replay dies at the domain gate, so no margin band can reach it.
Guard — interferers entering the tier	0 (fan, traffic, white, music, 4 real speech recordings — all clean)
Cost — additional healthy "possible" alarms	+8 of 140 (5.7%) — on top of the 6 confirmed FPs, healthy vehicles seeing an alarm would rise from 6 to 14 (4.3% $\rightarrow$ 10%)

6.1 Decision: NOT SHIPPED — rejected on measurement

The tier produced **zero measured recall benefit and a near-doubling of healthy-vehicle alarms**. Eight healthy engines would have been told "possible power steering / rocker / bearing issue" for nothing. The intuition behind the request is still served, in the two places it belongs to:

- **Same recording, degraded capture** $\rightarrow$ Tier 1 already matches far below "72% similarity" conditions (harsh speaker, 50 cm, noise) because coherence, not similarity, is the criterion.
- **Different vehicle, similar fault** $\rightarrow$ this is a *data* problem. The 65–72% embedding band is dominated by healthy engines on the current reference library; the band becomes usable exactly when the library contains diverse real recordings per family (see §5.6). Re-run `measure_near_match_tier.mjs` after each significant library expansion — the day its cost line drops near zero, the tier can ship with the same code that measured it.

7. Operating limits (honest)

- Tier 1 identifies **these exact recordings**. It will never recognise a different car's fault; that is Tier 2's job, and Tier 2's recall is data-bound (~72% held-out).
- All acoustic figures derive from **channel simulations**; the definitive speaker $\rightarrow$ air $\rightarrow$ microphone test runs on a physical device, and every session (including failures) now leaves remotely readable telemetry for exactly that purpose.
- One device-class risk stands open: OS-forced noise suppression, now visible per-session in `capture_settings`.

8. Harness map (run these, don't rebuild them)

Question	Harness
Tier 1 thresholds still safe?	<code>review_constellation_shipped.mjs</code> (re-run after ANY index change)
Tier 1 sensitivity across capture conditions	<code>proto_constellation_sensitivity.mjs</code>
Tier 2 operating point / FP baseline	<code>rca_healthy_fp_sweep.mjs</code> (all 140 — never a stride sample)
Gate behaviour under speaker coloration	<code>rca_gate_boundary.mjs</code> , <code>rca_gate_fix_validation.mjs</code>
Session-rule calibration	<code>rca_tiebreak_probe.mjs</code> , <code>rule_explorer.mjs</code>
End-to-end in the real app	fake-mic injection E2E (see handover doc §7.1)
Fast unit/policy assertions	<code>qa_unit_tests.mjs</code> , <code>qa_ethanol_feature.mjs</code>